IEEE Access
Multidisciplinary ⋮ Rapid Review ⋮ Open Access Journal

Date of publication xxxx 00, 0000, date of current version xxxx 00, 0000.
Digital Object Identifier 10.1109/ACCESS.2026.0000000

NMC-OPT: Intelligent Compiler Optimization for Near-Memory Computing

MALISETTY SAI NIKHIT[1], DHAYAPULE KARTIKESH RAO[1], METTA VENKATA SURYA SAI SRIKAR[1], AAHAAN KOTHARI[1], GOVIND BHATIA[1], and BHUVANESWARI M[1]
[1]Vellore Institute of Technology, Vellore, India

Corresponding author: Prof. Bhuvaneswari M (e-mail: m.bhuvaneswari@vit.ac.in).

ABSTRACT Near-Memory Computing (NMC) and Processing-in-Memory (PIM) reduce data movement by placing computation close to memory, which benefits memory-intensive workloads whose time and energy are dominated by operand transfer. A central difficulty is deciding whether a kernel should execute on the host CPU or on a near-memory unit, because transfer overhead, memory-access patterns, workload size, and available parallelism all affect the outcome. This paper presents NMC-OPT, a compiler-assisted framework that treats offloading as a cost-based decision. NMC-OPT characterizes a kernel, constructs an intermediate representation, estimates the end-to-end cost of CPU and near-memory execution with an analytical cost model, evaluates loop-transformation and bank-aware data-layout candidates, and recommends an execution strategy before run time. The prototype combines a React/TypeScript frontend with a Python/FastAPI backend and targets streaming and linear-algebra kernels such as Vector Add, Reduction, ReLU, GEMV, and Fused AXPY. For the example kernel (two million elements), the cost model estimates a near-memory latency of 0.77 ms against a host latency of 0.90 ms (a 1.16× speedup), a problem-size crossover near 700K elements, a 58% reduction in data-movement energy, and a link-bandwidth break-even near 48 GB/s; bank-interleaved placement lowers latency by about 26% relative to an unaligned layout. These results are analytical estimates from the prototype cost model; calibrating and validating them against hardware or cycle-accurate simulation is the primary direction for future work.

INDEX TERMS analytical cost modelling, compiler optimization, memory-bound kernels, near-memory computing, processing-in-memory, workload characterization

I. INTRODUCTION

MODERN computing applications increasingly process large arrays, tensors, and data structures. In such workloads, execution time and energy consumption can be dominated by moving operands between processing units and memory rather than by arithmetic operations alone. Near-Memory Computing (NMC) and Processing-in-Memory (PIM) address this limitation by placing processing capabilities close to memory banks or within memory devices. Recent research has explored different PIM architectures, programming models, and workload-mapping techniques to reduce data movement and improve system efficiency [1]–[4], [16].

The main problem addressed in this work is the difficulty of determining when a computational kernel should execute on a host CPU and when it may benefit from near-memory execution. Small workloads can perform poorly on an accelerator because launch and transfer overheads dominate, whereas large streaming workloads may benefit from higher effective memory bandwidth. Therefore, an effective compiler-assisted framework must consider end-to-end execution cost rather than device-side computation time alone.

Existing research has explored multiple approaches to memory-centric computation. Architecture-focused studies investigate DRAM-based processing, memory-side parallelism, and the integration of processing capabilities into high-bandwidth memory systems [1], [5]–[7], [19], [24]. Programming frameworks such as SimplePIM provide higher-level abstractions for distributing data and computation across processing-in-memory resources, reducing the burden of writing low-level device-specific code [14], [18]. Other studies examine compiler support and workload mapping to make PIM systems easier to program and use [10], [17], [26], [30].

A second research direction focuses on automatic compiler optimization for tensor and loop-based workloads. Frameworks such as TVM and Ansor demonstrate how intermediate representations, scheduling transformations, and automated search can improve generated programs across different hardware targets [15], [22]. TensorIR provides a structured representation for expressing tensor computations and transformations, while related compiler systems explore optimized tensor scheduling and dynamic workloads [21], [23], [25]. These approaches establish important principles for representing computations, generating optimization candidates, and evaluating alternative schedules.

Despite these advances, several limitations remain relevant to practical NMC offloading. Manually optimized libraries can deliver good performance for known workloads but may require significant effort to support new kernels [9], [20]. Instruction-level offloading approaches may be tied to particular hardware primitives or execution models [8], [11], [12]. Search-based optimizers can incur substantial exploration costs and may not provide a straightforward explanation of why a workload should execute on a CPU or a near-memory processing unit [13], [22].

Another limitation is the difficulty of comparing results obtained using different hardware platforms, simulation assumptions, and measurement procedures. Peak memory bandwidth alone does not determine application performance because

transfer overhead, memory-access regularity, bank conflicts, synchronization, and workload size can affect end-to-end execution time. NMC-OPT therefore exposes its cost assumptions and distinguishes analytical estimates from measured performance.

This paper proposes NMC-OPT, a compiler-assisted framework for cost-aware near-memory offloading. The framework analyzes source kernels, extracts loop and memory-access characteristics, constructs an intermediate representation, estimates CPU and NMC execution costs, and evaluates candidate transformations. Its modular design separates source validation, workload analysis, cost modelling, candidate evaluation, and reporting. The approach emphasizes program representation, optimization decisions, and cost-based evaluation rather than relying exclusively on hardware-specific manual tuning.

The remainder of this paper is organized as follows. Section II reviews 17 research works related to PIM architectures, programming frameworks, and compiler optimization, and closes with a synthesis of the research gap. Section III presents the proposed system architecture, methodology, modules, mathematical formulation, and algorithms. Section IV describes the implementation environment and reports the analytical performance results. Section V concludes the paper and discusses future directions. The references and required author declarations are presented at the end.

II. RELATED WORKS

A. MUTLU ET AL. — A MODERN PRIMER ON PROCESSING IN MEMORY

Proposed solution: The authors present a broad survey of processing-near-memory and processing-using-memory architectures. The paper explains how moving computation closer to memory can reduce data movement and improve performance and energy efficiency.

Methodology and tools: The work synthesizes research across memory technologies, processing architectures, applications, programming models, and system integration. It compares different PIM approaches and discusses their implications for programmability, compiler support, data placement, and system design.

Results and effectiveness: The survey establishes a cross-layer perspective on PIM and quantifies why data movement is an important bottleneck: it reports that data movement can account for more than 60% of system energy in some consumer workloads, and that bulk in-DRAM copy techniques such as RowClone achieve over 11× latency and 74× energy reductions for 4 KB copies. It provides a useful foundation for identifying workloads that may benefit from near-memory execution.

Research gap: The work is a survey and does not implement an automatic kernel-level cost model for CPU-versus-NMC decisions.

B. CHO ET AL. — MAIN-MEMORY ACCELERATORS FOR BANDWIDTH-BOUND INFERENCE

Proposed solution: This work investigates the acceleration of bandwidth-bound inference workloads using main-memory processing resources. It examines how computation can be moved closer to stored data to reduce memory-transfer overheads.

Methodology and tools: Cho et al. propose StepStone, a main-memory PIM design that accelerates the bandwidth-bound small-batch GEMMs of fully connected layers. Its GEMM dataflow respects the host's XOR-based DRAM address mapping so that each PIM unit operates only on locally mapped data, and processing units are placed at the channel, buffer-chip, and bank-group levels together with memory-side address-generation logic. The design is evaluated in simulation using Ramulator and gem5, with an Intel Xeon Platinum 8280 CPU (oneDNN) and an NVIDIA Titan Xp GPU (CUTLASS) as reference points.

Results and effectiveness: StepStone reports up to 12× lower batch-1 GEMM latency than the CPU at the bank-group level and up to 16× lower end-to-end inference latency, with the CPU and GPU overtaking it only at batch sizes of about 256 or more. These results support the idea that execution placement should depend on workload characteristics, particularly batch size and memory intensity, rather than on peak compute capability alone.

Research gap: A general-purpose compiler workflow for cost-based offloading of arbitrary C loop kernels remains outside its primary scope.

C. OLIVEIRA ET AL. — MIMDRAM

Proposed solution: MIMDram explores an end-to-end processing-using-DRAM system designed to support high-throughput multiple-instruction, multiple-data computation. It investigates how computation can be organized around the capabilities of DRAM-based processing resources.

Methodology and tools: MIMDRAM uses fine-grained DRAM to provide MIMD execution across DRAM mats with SIMD execution inside each row segment, allocating only the resources each operation needs. Programmer transparency is provided through LLVM compiler passes that auto-vectorize code and map it to processing-using-DRAM operations, and the system is evaluated in gem5 with CACTI-based area and energy models against Intel Skylake CPU and NVIDIA A100 GPU baselines.

Results and effectiveness: Relative to the state-of-the-art processing-using-DRAM baseline SIMDRAM, MIMDRAM reports about 34× higher performance and 14.3× better energy efficiency at an area overhead of roughly 1.1% of the DRAM chip, and about 30.6× better energy efficiency than the CPU. The work illustrates the importance of evaluating data placement, parallel execution, and compiler support together for memory-centric systems.

Research gap: It does not provide the same lightweight source-level workflow for comparing CPU and NMC execution costs and generating recommendations.

D. DEVAUX — THE TRUE PROCESSING-IN-MEMORY ACCELERATOR

Proposed solution: This work discusses the design and positioning of a practical processing-in-memory accelerator. It

examines the potential of computation close to memory to reduce data movement and improve application efficiency.

Methodology and tools: Devaux describes the UPMEM general-purpose PIM accelerator, in which standard DDR4 DRAM chips embed in-order 32-bit DPU cores. Each 4Gb chip integrates eight DPUs clocked at 500 MHz with per-DPU scratchpad memories and DMA, a DIMM carries 128 DPUs, and a typical server provides around 2,048 DPUs with roughly 2 TB/s of aggregate internal bandwidth. The analysis contrasts this near-data organization with conventional DRAM in terms of bandwidth, supported operations, and end-to-end behavior.

Results and effectiveness: The reported figures include roughly 20× application speedup and about 10× lower energy and total cost of ownership relative to an x86 server with standard DRAM, with the energy to move a 64-bit operand cited as about 150 pJ for PIM against about 3,000 pJ for conventional DRAM. These figures reinforce that system integration and per-operand data-movement cost, rather than peak bandwidth alone, determine the realized benefit.

Research gap: It does not implement the modular compiler analysis and kernel-specific cost-reporting workflow targeted by NMC-OPT.

E. KIM ET AL. — AQUABOLT-XL HBM2-PIM

Proposed solution: Aquabolt-XL demonstrates a high-bandwidth-memory processing-in-memory direction intended to support machine-learning and other data-intensive workloads. The approach integrates processing capabilities with high-bandwidth memory.

Methodology and tools: The system combines memory technology with processing functionality and examines how memory-side computation can be integrated into an accelerator platform. Its evaluation context focuses on supported operations, workload mapping, and the relationship between memory access and computation. The platform illustrates the importance of adapting software to the operations supported by the target device. Its design also shows that memory-side processing must be considered as part of a larger hardware and software system.

Results and effectiveness: Aquabolt-XL integrates a 16-wide FP16 processing unit at each bank and reports about 1.2 TFLOPS of embedded compute, roughly 2× system performance, and over 70% lower energy relative to standard HBM2. It supports the argument that memory-side acceleration is useful when workloads match the target's supported operations and data-access characteristics.

Research gap: A portable compiler-level cost model for arbitrary loop kernels is not the primary purpose of the hardware platform.

F. HYUN ET AL. — CHARACTERIZING A COMMERCIAL PIM TECHNOLOGY

Proposed solution: This work characterizes a commercial processing-in-memory technology to clarify its practical capabilities and limitations. It investigates how observed behavior differs from assumptions based solely on idealized specifications.

Methodology and tools: The methodology focuses on hardware characterization and workload performance. The evaluation studies system behavior under different execution conditions to identify factors that influence the benefits of PIM. By examining practical system characteristics, the work provides information that can inform the calibration of cost models. Such characterization is important when estimating effective bandwidth, execution overhead, and resource utilization.

Results and effectiveness: Using the uPIMulator compiler and cycle-level simulator, validated against the 16-workload PrIM suite, the study shows that design features such as 16-way SIMT and higher instruction-level parallelism yield average speedups of about 2.6× and 2.7× respectively over the baseline UPMEM DPU. It motivates the use of calibrated system parameters rather than treating peak internal bandwidth as a guaranteed application-level speedup.

Research gap: Hardware characterization does not itself provide an end-user source compiler that automatically recommends CPU or NMC execution.

G. GÓMEZ-LUNA ET AL. — MACHINE-LEARNING WORKLOADS ON MEMORY-CENTRIC SYSTEMS

Proposed solution: This research evaluates and characterizes machine-learning workloads on memory-centric computing systems. It investigates how workload properties influence performance when computation is moved closer to memory.

Methodology and tools: The study examines workload behavior under memory-centric execution and considers the interaction between computation, data movement, and available processing resources. Its evaluation emphasizes matching workload characteristics to the capabilities of the target system. The methodology supports the analysis of data-intensive workloads rather than relying exclusively on processor throughput. This is relevant to compiler optimization because workload characterization can guide decisions about execution placement.

Results and effectiveness: On a real general-purpose PIM system with over 2,500 cores, the study reports a decision-tree classifier running 27× faster than an eight-core Xeon CPU and 1.34× faster than an A100 GPU, and K-Means running 2.8× and 3.2× faster than the CPU and GPU respectively. It supports workload classification as an important prerequisite for effective PIM optimization.

Research gap: A unified compiler workflow for non-machine-learning C kernels, including cost estimation and interactive recommendations, is not its central deliverable.

H. HADIDI ET AL. — CAIRO

Proposed solution: CAIRO explores compiler-assisted instruction-level offloading for selected processing-in-memory operations. Its objective is to make PIM offloading more systematic by using compiler and workload information.

Methodology and tools: The approach uses compiler support to identify opportunities for executing selected operations on PIM resources. It considers workload behavior and memory-related characteristics to determine where offloading may be beneficial. The method is associated with a specific PIM execu-

tion model and examines the relationship between instructions, data locality, and execution placement. This provides a useful reference point for compiler-assisted offloading.

Results and effectiveness: CAIRO demonstrates that compiler support can reduce some of the burden associated with manual offloading. It also illustrates how information about program behavior can inform execution decisions.

Research gap: Its instruction-level scope differs from the whole-kernel cost comparison and multi-stage transformation workflow targeted by NMC-OPT.

I. IBRAHIM ET AL. — BALANCED DATA PLACEMENT FOR GEMV ACCELERATION

Proposed solution: This work studies balanced data placement for general matrix-vector multiplication (GEMV) using processing-in-memory resources. It focuses on distributing computation and data to improve the utilization of memory-side processing units.

Methodology and tools: The methodology considers how matrix and vector data are distributed across available processing resources. It evaluates the effect of data placement on workload execution and the utilization of memory-side parallelism. GEMV is relevant because its execution can be limited by memory bandwidth and data movement. The work therefore provides a useful example of how placement decisions influence performance.

Results and effectiveness: The proposed placement scheme, PIMnast, reports up to 6.6× GEMV speedup and up to 25.7× improvement over column-major placement, with up to about 3.5× end-to-end speedup for OPT-class models. This motivates including GEMV in NMC-OPT's benchmark suite and supports analyzing data distribution rather than evaluating computation in isolation.

Research gap: Its workload-specific focus does not replace a unified analysis pipeline that supports multiple kernel families.

J. GOGINENI ET AL. — SWIFTRL

Proposed solution: SwiftRL explores efficient reinforcement learning on real processing-in-memory systems. It investigates how reinforcement-learning workloads can be mapped to memory-side processing resources.

Methodology and tools: The study examines the characteristics of reinforcement-learning workloads and the practical requirements of executing them on PIM resources. Its evaluation considers the interaction between workload structure, available device capabilities, and execution behavior. The work's real-system orientation is important because it emphasizes the need to validate proposed optimizations against actual platform constraints. It also broadens the application context beyond simple streaming kernels.

Results and effectiveness: SwiftRL reports near-linear scaling and up to about 2.08× speedup over an Intel Xeon CPU and up to 4.84× over an NVIDIA RTX 3090 GPU for tabular Q-learning and SARSA on a real PIM system with over 2,500 cores. It illustrates the potential for memory-centric execution in additional application domains and reinforces the importance of workload-specific mapping and practical evaluation.

Research gap: A compiler front end for arbitrary loop nests and a transparent analytical CPU/NMC crossover model remain separate objectives.

K. SHIN ET AL. — IMTP

Proposed solution: IMTP explores search-based code generation for in-memory tensor programs. Its goal is to identify effective implementations by exploring different program and data-distribution choices. The work first appeared as an arXiv preprint under the title IMTP and was later published at ISCA 2025 as ATiM [13].

Methodology and tools: The methodology constructs a search space over tensor-program schedules and data-distribution strategies. Compiler representations are used to describe candidates, while evaluation ranks candidate implementations according to their expected or observed effectiveness. The approach is relevant to loop scheduling and target-aware optimization because it treats optimization as a systematic candidate-selection problem. It also illustrates how a compiler can automate decisions that would otherwise require manual tuning.

Results and effectiveness: IMTP demonstrates the potential of search-based optimization for in-memory tensor workloads. It supports the use of structured program representations and candidate evaluation.

Research gap: Search-based exploration may incur additional cost and does not inherently provide the compact analytical CPU-versus-NMC explanation emphasized by NMC-OPT.

L. CHEN ET AL. — SIMPLEPIM

Proposed solution: SimplePIM provides a software framework intended to make programming real processing-in-memory systems more productive. It offers higher-level abstractions for data partitioning, iteration, and communication.

Methodology and tools: The framework provides abstractions for distributing arrays and computations across processing units and managing communication between the host and PIM resources. Its evaluation includes applications executed on a real UPMEM system. The authors compare the framework with lower-level or hand-optimized implementations, focusing on programming effort and application performance. This provides a practical reference for understanding the benefits of higher-level PIM software abstractions.

Results and effectiveness: The work reports reductions in code size and application-dependent performance improvements. It demonstrates that a higher-level programming interface can improve productivity while retaining useful performance.

Research gap: SimplePIM primarily focuses on programming abstraction rather than automatic cross-target, cost-based execution recommendations.

M. CHEN ET AL. — TVM

Proposed solution: TVM presents an end-to-end optimizing compiler for deep-learning workloads. It combines program representations, scheduling, and code generation to support optimized execution across different hardware backends.

Methodology and tools: TVM uses graph-level and operator-level representations to express computations and optimization opportunities. Its scheduling infrastructure supports transformations that influence loop organization, memory access, and generated code. The framework evaluates optimized programs across different workloads and hardware targets. This establishes an important precedent for modular compiler infrastructure and target-aware optimization.

Results and effectiveness: TVM demonstrates that compiler-driven scheduling and code generation can improve the performance and portability of machine-learning workloads. It provides a foundation for later tensor compiler research.

Research gap: General tensor optimization does not automatically account for NMC-specific transfer overhead, bank layout, and CPU-versus-NMC execution decisions.

N. ZHENG ET AL. — ANSOR

Proposed solution: Ansor generates high-performance tensor programs through automated schedule search. It aims to discover effective implementations without requiring programmers to manually specify every optimization decision.

Methodology and tools: Ansor constructs candidate tensor schedules and explores them using a search strategy guided by cost evaluation. The system uses structured program representations to generate alternative schedules and evaluate their potential. Its evaluation spans tensor workloads and hardware targets, illustrating the role of automated search in discovering optimizations that may be difficult to write manually.

Results and effectiveness: Ansor demonstrates the potential of automated schedule generation to identify high-performance tensor programs. It shows how search strategies can support compiler optimization.

Research gap: Search-based tuning can incur exploration costs and does not inherently provide an explicit analytical explanation of the CPU-versus-NMC crossover.

O. FENG ET AL. — TENSORIR

Proposed solution: TensorIR introduces a tensor-program abstraction designed to expose transformations for automatic optimization. It provides a structured representation of loops, buffers, and computational blocks.

Methodology and tools: TensorIR represents tensor computations in a form that makes transformations explicit and composable. The representation is integrated with the TVM compiler ecosystem and supports the expression and evaluation of optimized tensor programs. The approach emphasizes compiler representations that preserve useful structural information while exposing optimization opportunities. This is relevant to NMC-OPT's use of an intermediate representation for workload analysis and candidate evaluation.

Results and effectiveness: TensorIR provides a structured basis for tensor-program optimization and supports systematic transformations. It motivates making loop structure and memory accesses explicit before evaluating alternatives.

Research gap: TensorIR is a compiler abstraction rather than a complete NMC offloading policy with a calibrated end-to-end cost model.

P. KHAN ET AL. — CINM (CINNAMON)

Proposed solution: CINM (Cinnamon) is an MLIR-based compilation infrastructure that targets both compute-in-memory and compute-near-memory paradigms through a hierarchy of dialects, raising programmability by lowering a device-agnostic representation to device-specific back ends.

Methodology and tools: Built on MLIR, CINM accepts inputs through linalg, TOSA, and torch front ends and lowers them to a device-agnostic cinm dialect and then to cim and cnm abstractions. It targets a real UPMEM system (16 DIMMs) for compute-near-memory and a simulated PCM memristor accelerator in gem5 for compute-in-memory, evaluated on the PrIM benchmarks with an ARMv8-A host baseline.

Results and effectiveness: For compute-in-memory, CINM reports roughly 10× geometric-mean speedup (up to 30× with optimizations) and about 5× energy reduction over the ARM host, and its generated UPMEM code is up to 2× faster than hand-optimized PrIM while using roughly 15× less source code.

Research gap: CINM provides a general compilation infrastructure across in-memory and near-memory targets, but does not focus on an explicit analytical CPU-versus-NMC crossover decision made before execution, which is the specific concern of NMC-OPT.

Q. SHIN ET AL. — PIMFLOW

Proposed solution: PIMFlow is a compiler and runtime system that executes CNN inference on DRAM-based PIM using software only, creating inter-node parallelism through graph transformation and splitting work between the GPU and PIM.

Methodology and tools: PIMFlow applies graph transformations to expose task- and data-parallelism and uses a scheduler to explore execution options across the GPU and PIM. It is evaluated against GPU-only execution on an NVIDIA RTX 2080 Ti using Accel-Sim and GPGPU-Sim for the GPU and Ramulator for the near-bank PIM, modelling a Newton-style PIM architecture.

Results and effectiveness: PIMFlow reports up to 82% end-to-end speedup and about 26% average energy reduction for CNN inference relative to GPU-only execution.

Research gap: PIMFlow targets CNN graph partitioning between GPU and PIM rather than a general, source-level cost-based decision between host-CPU and near-memory execution for arbitrary loop kernels.

R. SYNTHESIS AND RESEARCH GAP

Across these works, two complementary directions emerge. Architecture and characterization studies establish that the benefit of near-memory execution depends on workload size, access regularity, and effective internal bandwidth rather than on peak bandwidth alone, while compiler and programming-framework studies provide the representations, scheduling, and search needed to generate efficient implementations. What remains comparatively underdeveloped is a lightweight, source-level mechanism that decides, before execution and with a transparent analytical rationale, whether a given kernel should run on the host or near memory, and that reports the problem-size

threshold at which this choice changes. NMC-OPT addresses this gap by combining workload characterization, an analytical CPU-versus-NMC cost model, and bank-aware data-layout reasoning into a single compiler-assisted decision procedure.

III. PROPOSED WORK / SYSTEM

A. PROBLEM FORMULATION

NMC-OPT addresses the problem of selecting an appropriate execution strategy for computational kernels whose performance is influenced by data movement. Conventional CPU execution may be suitable for small workloads, irregular computations, or kernels with substantial data reuse. Near-memory execution may be beneficial for larger, memory-intensive workloads when its available bandwidth and parallel processing resources outweigh transfer, launch, and synchronization overheads. The execution decision must therefore consider the workload size, number of operations, memory traffic, access regularity, and target-system parameters.

The proposed system treats offloading as a cost-based compiler decision. Given a source kernel, input size, and target configuration, the framework extracts relevant program characteristics and estimates the end-to-end costs of CPU and NMC execution. It then evaluates supported transformation candidates, such as loop tiling and bank-aware data distribution, and recommends a strategy that minimizes the estimated cost while respecting correctness and resource constraints. Since the current prototype is a compiler-assistance tool, its recommendations must be distinguished from actual execution on physical PIM hardware.

The current formulation treats execution placement as a binary choice between host and near-memory execution for an entire kernel. Partial offloading, in which a kernel is split so that part of its iterations execute on the host and part near memory, is not considered in this work. Supporting it would require the cost model to account for the additional synchronization and intermediate data-exchange costs between the two partitions, and it is a natural extension of the present formulation.

The framework's objective can be stated as a measurable target: for a representative set of kernel and input-size combinations, the recommended execution strategy should match the strategy with the lower measured or cycle-accurately simulated end-to-end cost, and the predicted crossover input size should fall within a small tolerance of the measured crossover. Quantifying how often this target is met is the subject of the validation study identified as future work.

B. PROPOSED SYSTEM ARCHITECTURE

As shown in Fig. 1, the architecture begins with Tier 1: Client/User Interaction Layer, featuring the Web Dashboard (React/TS/Vite) containing views such as AnalyticsView, ChatView, PipelineView, SimulatorView, and WorkbenchView, alongside an Interactive CLI for automation and script integration. The gateway layer handles routing and security via Tier 2: Gateway & Schema Validation Layer, which encompasses FastAPI endpoints and Pydantic-based request/response schema validation (`schemas.py`). The core processing logic is housed in Tier 3: System Core Backend Layer, which coordinates the Intelligent Optimizer (`optimizer.py`) for finding optimal constraints, the Analyzer (`analyzer.py`) for performance and parameter tuning, the NMC Cost Model (`cost_model.py`) for mathematical modelling and estimations, the NMC Benchmarks Suite (`benchmarks.py`) for workload execution and tracking, and the LLM Copilot / Assistant (`copilot.py`) for reasoning and interactive suggestions. Finally, Tier 4: Data, Compilation, and IR Layer separates underlying data assets into the Data & Intermediate Representation (IR) Layer (comprising Repository & Configuration and the Intermediate Representation Compiler `ir.py` for system specs, input variables, and file outputs) and the Research Suite & Evaluation module for visualization, plotting, walkthrough decks, and report generation. The copilot provides natural-language explanations of the analysis and does not change the recommendation itself; the execution-strategy decision is produced deterministically by the analytical cost model (Algorithm 1) and is therefore reproducible. In this context, the word intelligent refers to the automated kernel characterization, cost-based reasoning, and natural-language explanation provided by the framework rather than to a learned decision policy.

C. DETAILED METHODOLOGY

1) Source Processing and Kernel Analysis

The first stage accepts a supported computational kernel and validates its structure. The kernel is analyzed to identify loop bounds, array accesses, data dependencies, and computational operations. This information is used to estimate the amount of computation and data movement associated with the workload. Compiler abstractions that represent tensor computations and their transformations provide useful context for this stage [15], [23]. For NMC-OPT, the analysis produces the loop bounds, access patterns, and dependence information needed to decide whether the workload can be represented in the supported intermediate form; kernels that cannot are executed on the host.

2) Intermediate Representation and Polyhedral Lowering

The intermediate representation (IR) provides a structured representation of the computation that can be analyzed independently of the original source syntax. For supported affine loop nests, polyhedral methods can represent iteration domains and access relationships mathematically, enabling transformations such as loop tiling and loop reordering. The lowering stage applies loop fusion and loop tiling to supported affine loop nests (Section III-C.7) to improve locality and expose parallelism; each transformation is applied only if it preserves data dependences and a valid execution order. Compiler infrastructures such as MLIR and tensor-optimization frameworks such as TVM provide related foundations for representing and transforming computational programs [15], [23], [25].

3) Analytical Cost Modelling

The cost model estimates the time required to execute a kernel on the host and near-memory processing units. It considers computational work, memory traffic, effective bandwidth, transfer latency, and synchronization overhead. The model

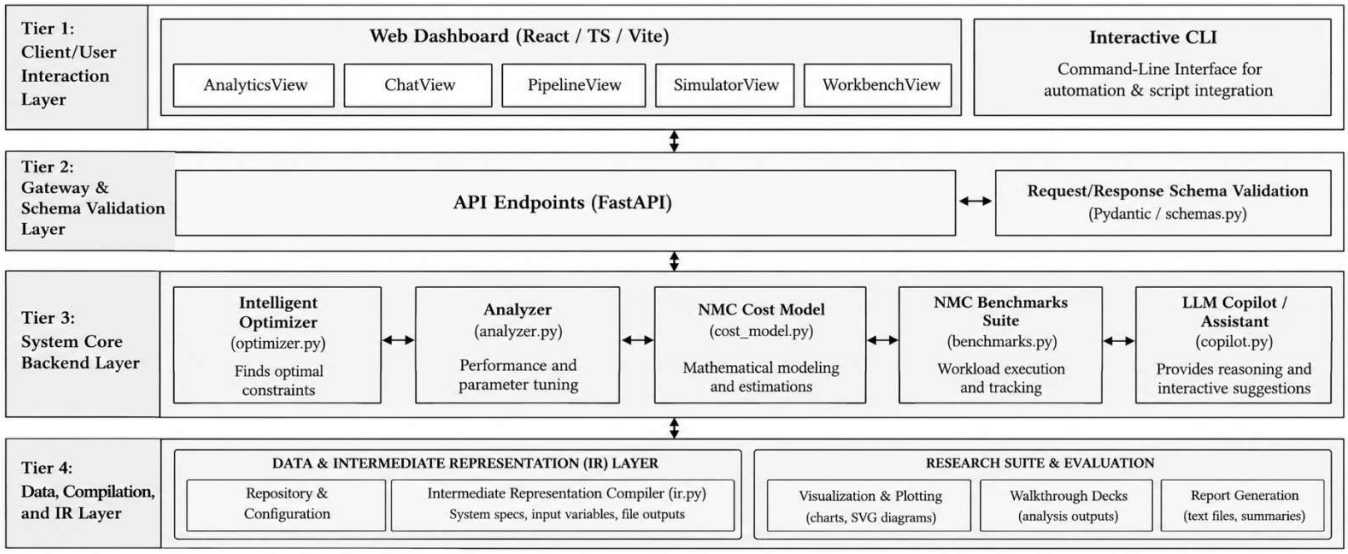


FIGURE 1. NMC-OPT system architecture.

is used to identify the conditions under which near-memory execution is expected to outperform host execution.

Let T_{CPU} denote the predicted host execution time and T_{NMC} denote the predicted total time for near-memory execution. The compiler selects the strategy with the lower estimated cost, subject to correctness and hardware constraints.

4) Execution Strategy Selection

The strategy-selection stage compares the predicted costs and chooses between host execution and near-memory execution. The decision considers input size and the relative contributions of computation, data transfer, and memory access. The purpose of this stage is to prevent the compiler from treating every kernel as an offloading candidate. PIM research and compiler-assisted offloading studies show why workload properties and execution overhead must be considered when selecting offload candidates [7], [8], [14].

5) Memory Bank Striping and Data Placement

The memory-layout stage distributes data across the available memory banks according to the selected layout strategy. NMC-OPT uses a 16-bank striping configuration to improve the distribution of accesses and reduce channel imbalance. The effectiveness of bank-level striping depends on the actual memory organization, access pattern, alignment requirements, and supported transfer granularity. Data placement has also been investigated in workload-specific PIM optimization [9]. In NMC-OPT, the proposed layout is evaluated against an unaligned naive-DRAM baseline layout (Fig. 13) so that its benefits can be attributed to the striping configuration rather than to other factors.

6) Kernel Generation and Evaluation

After selecting the execution strategy and applying the supported transformations, the framework produces the corresponding host or near-memory execution representation. The resulting implementation is evaluated using execution time, speedup, data-movement energy estimates, and other applicable metrics. The framework records the input size, selected strategy, transformation configuration, execution environment, and the estimated metrics for each experiment. These records support reproduction of the results and, once measured values are available, comparison of the predicted and observed preferred strategy.

7) SSA-Based IR, Loop Fusion and Loop Tiling

The intermediate representation is in static single-assignment (SSA) form: every value is defined exactly once, loop-carried values are merged by phi nodes, and memory accesses appear as explicit address computations followed by loads and stores, as in the Vector Add listing of Section III-H. This form makes the operations per iteration, the bytes moved per iteration, and the dependences between statements directly readable, which is what the feature-extraction and cost-model stages consume.

Two loop transformations operate on this IR. **Loop fusion** merges adjacent element-wise loops that share an iteration domain, so that an intermediate array is kept in a register instead of being stored and reloaded; this lowers the bytes moved per iteration and is the transformation that benefits ReLU in Section IV-C. **Loop tiling** partitions the iteration space into blocks of TILE iterations and assigns consecutive elements of a block to successive memory banks (Fig. 4), which is the transformation applied to GEMV when its tiling-eligibility test passes. Both transformations are applied only when the dependence test of Section III-H shows that no read-after-write, write-after-read, or write-after-write dependence is reversed; kernels with a loop-carried dependence on an accumulator, such as Reduction, are not fused or reordered.

D. MODULES AND THEIR FUNCTIONS

Table 1 summarizes the principal modules and their expected outputs.

As listed in Table 1, each module maps to a distinct stage

TABLE 1. NMC-OPT Modules and Their Functions

Module	Main function	Expected output
Input validation	Checks the submitted kernel and configuration	Validated input
Kernel analysis	Extracts loop, memory-access, and workload characteristics	Kernel profile
IR construction	Creates an intermediate representation	Analyzable computation
Cost modelling	Estimates host and near-memory execution costs	Cost estimates
Strategy selection	Chooses the lowest-cost valid execution strategy	Execution decision
Polyhedral optimization	Applies supported loop transformations	Optimized loop representation
Memory-layout optimization	Distributes data across memory banks	Transformed data layout
Code generation	Produces the selected execution representation	Host or near-memory code
Evaluation and visualization	Collects and displays performance information	Metrics, graphs, and reports

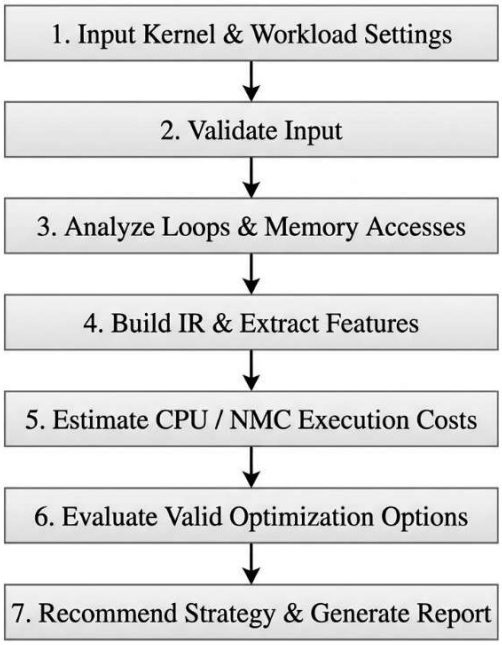


FIGURE 2. NMC-OPT execution workflow.

of the pipeline and produces a specific intermediate artifact—from a validated input and kernel profile, through cost estimates and an execution decision, to the final generated code and visual reports—so that the framework's behaviour can be traced and evaluated stage by stage.

E. WORKFLOW AND FLOW DIAGRAM

The workflow begins with a kernel and its input configuration. After validation, the framework analyzes the computation and constructs the intermediate representation. The cost model then estimates the execution costs for the available strategies. If near-memory execution is not predicted to be beneficial, the compiler selects the host path. Otherwise, it applies the supported transformations and generates the near-memory execution representation (Fig. 2).

Both paths lead to metric collection and reporting. Comparing predicted costs with measured execution times allows the framework to evaluate the quality of its strategy-selection mechanism. This workflow is consistent with the broader objective of integrating compiler analysis and performance-driven execution selection [8], [14], [17].

F. MATHEMATICAL FORMULATION

The following equations define a conceptual cost model for NMC-OPT. The parameter values used in this paper are listed in Section III-H; they are platform parameters that can be recalibrated for another target.

$$T_{\text{CPU}} = T_{\text{compute,CPU}} + T_{\text{memory,CPU}} + T_{\text{overhead,CPU}} \quad (1)$$

Here, $T_{\text{compute,CPU}}$ is the host computation time, $T_{\text{memory,CPU}}$ represents memory-related delay, and $T_{\text{overhead,CPU}}$ includes other relevant host-side overheads.

$$T_{\text{NMC}} = T_{\text{launch}} + T_{\text{transfer}} + T_{\text{compute,NMC}} + T_{\text{layout}} + T_{\text{sync}} \quad (2)$$

The total near-memory time includes kernel-launch overhead, data-transfer time, near-memory computation time, and synchronization overhead. Following Fig. 9, the evaluation adds the data-layout conversion cost T_{layout} to the right-hand side of (2); each overhead is counted once.

$$T_{\text{transfer}} = \frac{D_{\text{in}}}{B_{\text{in}}} + \frac{D_{\text{out}}}{B_{\text{out}}} \quad (3)$$

Here, D_{in} and D_{out} are the input and output data volumes, while B_{in} and B_{out} are the corresponding effective transfer bandwidths.

$$S^* = \arg\min_{S \in \mathcal{S}} \hat{T}(S) \quad (4)$$

Here, S denotes an execution strategy and $\hat{T}(S)$ denotes its predicted total execution time. The selected strategy minimizes predicted cost among the supported alternatives.

$$\text{Speedup} = \frac{T_{\text{baseline}}}{T_{\text{optimized}}} \quad (5)$$

A speedup greater than 1 indicates that the optimized implementation is faster than the specified baseline. Baseline and optimized values are computed under the same model parameters.

$$I = \frac{F}{D} \quad (6)$$

Here, F is the number of floating-point operations and D is the number of bytes transferred. Arithmetic intensity helps characterize whether a kernel is likely to be limited by computation throughput or memory bandwidth.

$$B_{\text{eff}} = \frac{D}{T_{\text{memory}}} \quad (7)$$

Here, D represents the measured or estimated data volume and T_{memory} represents the corresponding memory-transfer interval.

$$\text{Energy reduction} = \frac{E_{\text{baseline}} - E_{\text{optimized}}}{E_{\text{baseline}}} \times 100\% \quad (8)$$

Here, E_{baseline} and $E_{\text{optimized}}$ represent energy consumption for the baseline and optimized configurations, respectively. Both energies are estimated with the same data-movement boundary and per-byte method (Section IV-D.4).

These equations establish a basis for comparing execution strategies, characterizing memory behavior, and reporting performance. The cost-model terms are analytical and use the platform parameters of Section III-H; the resulting values are estimates, not measured performance.

G. ALGORITHMS

1) Algorithm 1: Cost-Based Execution Strategy Selection

Input: kernel profile K, input size n, target configuration Θ
Output: selected strategy S^* and estimates (T_{CPU}, T_{NMC})

procedure SELECTSTRATEGY(K, n, Θ)
  VALIDATEKERNEL(K) ▷ *supported operations only*
  $F \leftarrow \text{FlopCount}(K,n)$; $D_{in}, D_{out} \leftarrow \text{Traffic}(K,n)$
  $T_{CPU} \leftarrow F/P_{host} + (D_{in} + D_{out})/B_{host}$
  $T_{xfer} \leftarrow D_{in}/B_{in} + D_{out}/B_{out}$
  $T_{NMC} \leftarrow T_{launch} + T_{xfer} + T_{compute,NMC} + T_{layout} + T_{sync}$
  if NMCSUPPORTED(K) **and** $T_{NMC} < T_{CPU}$ **then**
    $S^* \leftarrow$ NMC
  else
    $S^* \leftarrow$ CPU
  end if
  return (S^*, T_{CPU}, T_{NMC})
end procedure

This algorithm formalizes the execution-placement decision. It avoids assuming that near-memory execution is always beneficial and instead selects the strategy with the lowest estimated cost. The decision follows the motivation of compiler-assisted PIM offloading [8], [14].

2) Algorithm 2: Memory-Aware Kernel Optimization

Input: validated kernel K, IR, bank configuration B (16 banks)
Output: transformed representation R^* and selected layout L^*

procedure OPTIMIZEKERNEL(K, IR, B)
  IR $\leftarrow$ BUILDIR(K)
  $(nests, acc) \leftarrow$ IDENTIFYLOOPNESTS(IR)
  $\mathscr{T} \leftarrow$ APPLICABLETRANSFORMS($nests, acc$) ▷ *tiling, fusion*
  $\mathscr{L} \leftarrow$ GENERATELAYOUTS(acc, B) ▷ *bank mappings*
  $best \leftarrow \infty$; $R^* \leftarrow \varnothing$; $L^* \leftarrow \varnothing$
  for all $t \in \mathscr{T}$ **do**
    for all $L \in \mathscr{L}$ **do**
      if VALID(t, L) **then** ▷ *dependences preserved*
        $c \leftarrow$ ESTIMATECOST(APPLY(IR, t, L))
        if $c < best$ **then**
          $best \leftarrow c$; $R^* \leftarrow$ APPLY(IR, t, L); $L^* \leftarrow L$
        end if
      end if
    end for
  end for
  return (R^*, L^*)
end procedure

This algorithm gives the optimization sequence applied to each kernel: candidates are generated, filtered by a legality test that preserves program semantics, and ranked by estimated cost. Search-based tensor compilers provide relevant context for exploring transformation candidates [21], [22], [23].

3) Properties of the Algorithms

Both algorithms are deterministic and inexpensive relative to the workloads they manage. Algorithm 1 evaluates two closed-form expressions and one comparison, so its decision cost is constant in the input size N once the kernel profile (F, D) is known, and that profile is obtained in a single pass over the intermediate representation. Algorithm 2 visits each candidate loop nest once and enumerates a small, fixed set of layout and transformation candidates, so its cost grows linearly with the number of loop nests and candidates but does not depend on the data size.

Correctness of the optimization step rests on the legality test in Algorithm 2: a candidate is retained only if the dependence analysis shows that no read-after-write, write-after-read or write-after-write dependence is reversed, so every retained candidate computes the same result as the original loop. Among the legal candidates the lowest estimated cost is selected, and Algorithm 1 chooses host execution whenever near-memory execution is unsupported or not cheaper, so the compiler never selects a strategy that its own cost model predicts to be slower than the host baseline.

The components described above are illustrated in the following figures: the overall compiler pipeline (Fig. 3), the sixteen-bank data-striping scheme applied by the memory-layout stage (Fig. 4), and the cost-based decision between host and near-memory execution as a function of input size (Fig. 5).

As illustrated in Fig. 3, the NMC-OPT compiler pipeline proceeds through nine stages: source-code ingestion, source parsing to an abstract syntax tree (AST), kernel analysis over the control-flow graph (CFG), intermediate-representation construction, analytical cost modelling, execution-strategy selection, polyhedral transformation of the geometric iteration space, code generation, and a closing evaluation-and-feedback step. The feedback edge returns evaluation results to the cost-modelling and strategy-selection stages, so that later decisions can be refined from earlier measurements.

Fig. 4 shows the conceptual mapping used by the memory-layout stage, in which successive input elements A[0], A[1], A[2], … are distributed across sixteen logical banks so that consecutive accesses fall on different banks. Interleaving the array in this way is intended to spread concurrent requests across banks, improving within-bank locality and reducing the channel imbalance and bank-conflict stalls that a contiguous single-bank layout would incur.

In this mapping, element i resides in bank (i mod 16) at row floor(i/16), so a sequential stream of sixteen accesses touches sixteen different banks and can be serviced concurrently, whereas an unaligned layout concentrates consecutive accesses on fewer banks and serializes them. The value of sixteen banks is a configuration parameter of the evaluation (Table 2) rather than a fixed property of the method, and the same rule generalizes to any bank count by replacing 16 with that count. The stall penalty that results when this mapping is not applied is quantified in Fig. 13.

Fig. 5 expresses the cost-based decision as the signed difference $\Delta T = T_{\text{CPU}} - T_{\text{NMC}}$ against input size. The difference is negative below about 700K elements, where the host is cheaper, and positive above it, where near-memory execution is cheaper; the sign change is the decision implemented by Algorithm 1. The worked example below shows the inputs to this decision for one benchmark kernel.

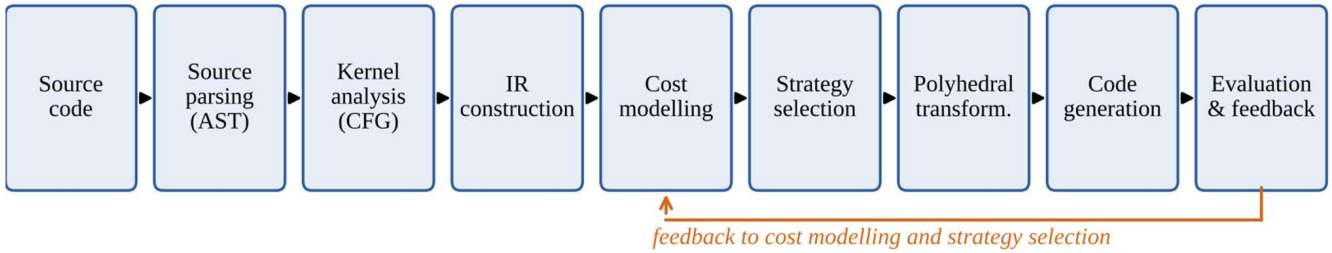


FIGURE 3. Compiler optimization pipeline.

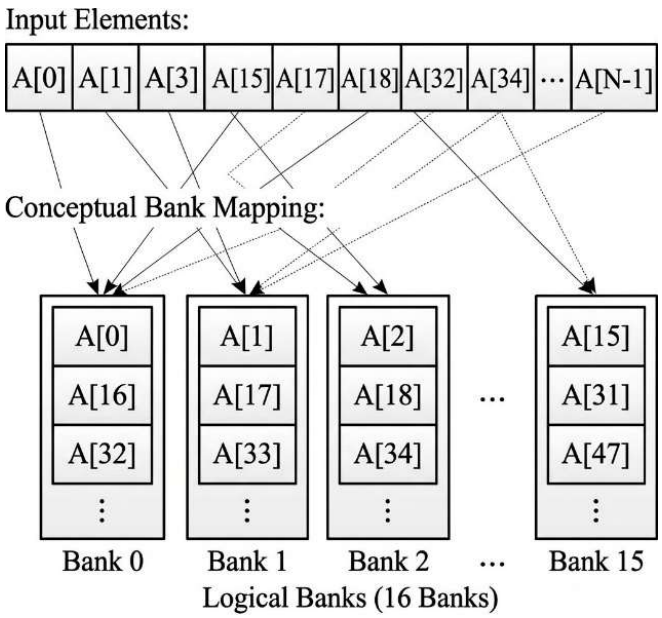


FIGURE 4. Memory-bank striping.

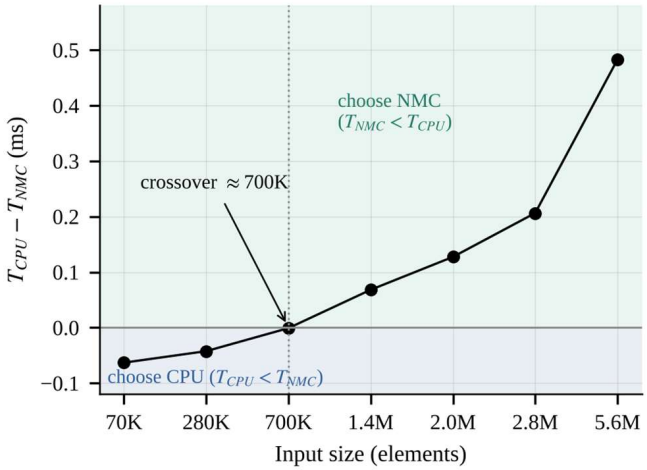


FIGURE 5. Cost-based offloading decision.

H. WORKED EXAMPLE: VECTOR ADDITION

To make the compiler stages concrete, consider the Vector Add kernel of the benchmark suite. The source kernel is shown first, followed by its SSA-form intermediate representation, the workload features extracted from it, and the transformed loop produced by the layout stage.

```
void vector_add(const float* A, const float* B, float* C, int N) {
  for (int i = 0; i < N; ++i)
    C[i] = A[i] + B[i];
}
```

```
entry: vector_add(ptr %A,%B,%C, i32 %N)
cond:  %i = phi [0,entry], [%i.next,inc]
       %c = icmp slt i32 %i, %N
       br %c, label body, label exit
body:  %pa = gep float, ptr %A, %i
       %a = load float, ptr %pa
       %pb = gep float, ptr %B, %i
       %b = load float, ptr %pb
       %r = fadd float %a, %b
       %pc = gep float, ptr %C, %i
       store float %r, ptr %pc
inc:   %i.next = add i32 %i, 1
exit:  ret void
```

From this representation, NMC-OPT extracts the workload features used by the cost model:

```
iterations : N
mem / iter : 2 loads + 1 store
bytes / iter: 3 x 4 B = 12 B
flops / iter: 1
D = 12N bytes, F = N flops
AI = F / D = 1/12 = 0.083 FLOP/B => memory-bound
```

The arithmetic intensity of 0.083 FLOP/byte places the kernel in the memory-bound regime, matching the roofline analysis of Fig. 11 and explaining why near-memory execution is beneficial. Because the kernel is element-wise and carries no loop-carried dependence, loop tiling and bank-interleaved placement are always legal; the transformed kernel distributes consecutive elements across the sixteen banks, as sketched below.

```
for (t = 0; t < N; t += TILE)
  for (i = t; i < min(t+TILE, N); ++i) {
    bank = i mod 16;   // stripe
    C[i] = A[i] + B[i];
  }
```

More generally, NMC-OPT applies a transformation only when it preserves the program's data dependences. For the affine loop nests targeted here, legality is checked by verifying that the transformed iteration order does not reverse any read-after-write, write-after-read, or write-after-write dependence on the same memory location. Element-wise kernels such as Vector Add and ReLU carry no such dependence and are always safe to tile and stripe, whereas Reduction carries a dependence on its accumulator that restricts reordering. Equivalence of a transformed kernel to the untransformed reference follows from the dependence test above; output-level comparison against the reference on representative inputs is ongoing (Section IV-A).

The same features instantiate the cost model of Section III-F. The compute term is the operation count divided by the effective compute throughput of the target (here $F = N$ flops); the memory term is the data volume divided by the effective bandwidth (Eq. 7, with $D = 12N$ bytes); and the transfer term is the input and output volumes divided by the interconnect bandwidths (Eq. 3). The throughput, bandwidth, and latency constants are platform parameters; in the prototype they take the values host compute 45 GFLOP/s and effective bandwidth 28 GB/s, near-memory compute 32 GFLOP/s and internal bank bandwidth 160 GB/s, host-to-device link 64 GB/s with 0.02 ms transfer-setup overheads, layout-conversion bandwidth 48 GB/s, and a 0.01 ms synchronization cost per barrier; these are the quantities that must be recalibrated for a specific target.

IV. IMPLEMENTATION AND RESULT ANALYSIS

A. IMPLEMENTATION TOOLS AND EXPERIMENTAL CONFIGURATION

NMC-OPT's design draws on established tools for near-memory evaluation, namely MacSim, VaultSim/DRAMSim2, the Structural Simulation Toolkit (SST), and Apache TVM, and the prototype itself is implemented as a FastAPI/Vite application. These tools serve different purposes: microarchitecture and memory-system simulation, compiler-based optimization, and application-level interaction. MacSim, DRAMSim2, and SST are relevant to architectural evaluation, while TVM provides a framework for compiler-based optimization [15], [27], [28], [29].

In the current prototype, the quantitative values reported in this paper are produced by NMC-OPT's own analytical cost model rather than by executing the architectural simulators; the simulators and TVM inform the structure and parameters of the cost model. Integrating calibrated simulator or hardware measurements, together with a complete record of the host configuration, memory parameters, DPU count, input sizes, and measurement procedure, is part of ongoing work to validate and reproduce the estimates. The analyzer, cost model, and full analysis pipeline are covered by unit tests, which verify kernel-fact extraction for Vector Add, Reduction, and GEMV, the cost-model behaviour, the CPU-wins-under-high-overhead and near-memory-wins-under-bandwidth-bound cases, and end-to-end pipeline execution including GEMV tiling eligibility. A systematic evaluation of the semantic correctness of transformed kernels against reference outputs remains ongoing.

B. IMPLEMENTATION SCREENSHOT

Fig. 6 shows the NMC-OPT Studio interface, in which a benchmark kernel (here Vector Addition) is selected, its canonical C source and SSA-based intermediate representation are inspected, the workload size is configured, and the compiler's optimization verdict is reported, including the recommended strategy, arithmetic intensity, crossover point, memory traffic, and estimated speedup. This interface is the interactive front end through which the analyses and graphs in this section are produced.

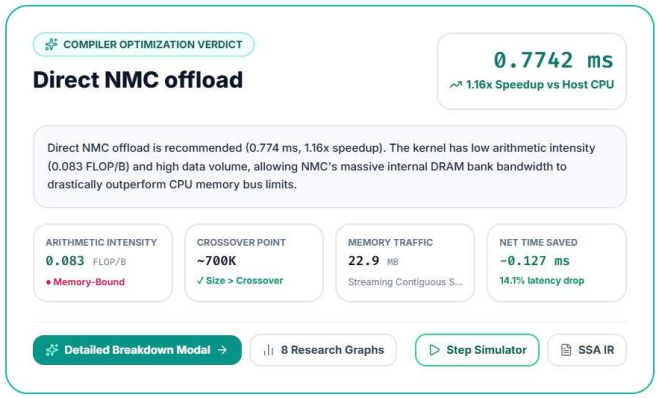


FIGURE 6. NMC-OPT Studio implementation interface.

TABLE 2. Experimental Parameters

Parameter	Value	Purpose
Memory-bank configuration	16 banks	Evaluate bank-level data distribution
Input-size range	70K to 5.6M elements	Evaluate scaling and crossover behavior
Example kernel size	2.0M elements (D = 24 MB)	Latency, energy and layout analysis
Host model	45 GFLOP/s, 28 GB/s	Host compute and memory ceilings
Near-memory model	32 GFLOP/s, 160 GB/s internal	Near-memory compute and bank-bandwidth ceilings
Host-to-device link	64 GB/s (default)	Transfer term in Figs. 7–9
Interconnect bandwidth sweep	16–128 GB/s	Evaluate transfer-bandwidth sensitivity
DPU scaling range	64–2048 DPUs	Evaluate parallel scaling

TABLE 3. Reported Analytical Results

Quantity	Value	Description
Crossover	Approx. 700K elements	Host and near-memory costs are equal
Host-only latency	0.9016 ms	Host baseline (2.0M elements)
Direct NMC latency	0.7742 ms	Near-memory latency (2.0M elements)
Speedup	1.16×	0.9016 / 0.7742
Break-even link bandwidth	Approx. 48 GB/s	Offload equals host latency
DPU sweet spot	256–512 DPUs	Efficiency of 96% or more
Host energy	4.80 mJ	200 pJ/B × 24 MB (DDR)
Direct NMC energy	2.02 mJ	84 pJ/B × 24 MB (TSV + link)
Interleaved NMC energy	1.68 mJ	70 pJ/B × 24 MB
Energy reduction vs host	58% / 65%	Direct / interleaved NMC

C. PARAMETER SETUP

Table 2 lists the parameters that define the evaluation, and Table 3 summarizes the corresponding analytical results.

Table 3 collects the analytical results obtained with these parameters, including the crossover near 700K elements, the example-kernel latencies of about 0.90 ms on the host and 0.77 ms near memory, the break-even link bandwidth near 48 GB/s, the parallel-efficiency sweet spot of 256–512 DPUs, and the data-movement energy estimates. These values are interpreted

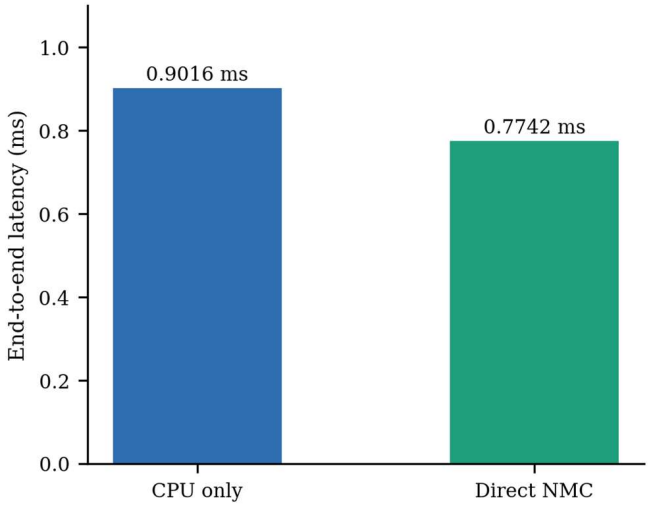


FIGURE 7. End-to-end execution latency comparison.

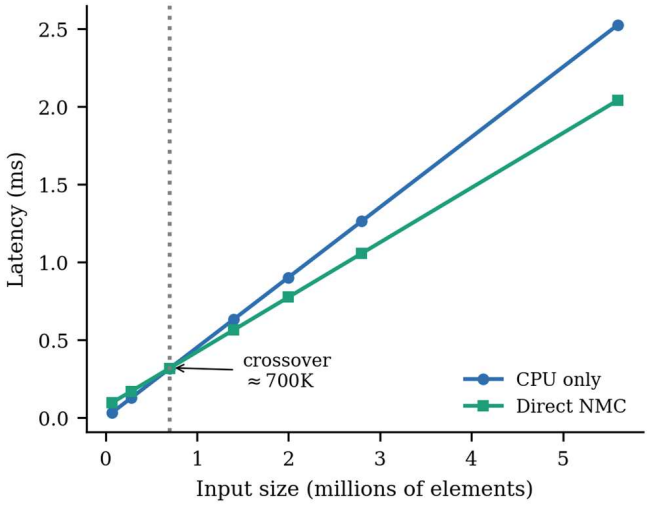


FIGURE 8. Execution latency as a function of input size.

in detail in the following subsections.

Applying the model to the full benchmark suite at the default input sizes gives direct-near-memory speedups of 1.16× for Vector Add (two million elements), 1.23× for Reduction (four million), 1.33× for Fused AXPY (two million), and 1.39× for element-wise multiply (two million). ReLU reaches 1.50× once loop fusion is applied, and GEMV, for which direct offload is marginally slower than the host (0.96×), reaches 1.50× with bank tiling. The estimated data-movement energy saving is approximately 58% across the suite (65% with bank-interleaved placement). All of these are analytical cost-model estimates rather than hardware or cycle-accurate-simulation measurements.

D. PERFORMANCE EVALUATION METRICS

1) End-to-End Latency and Speedup

End-to-end latency measures the time required to complete a workload, including the overheads included in the declared measurement boundary. For the example kernel ($N = 2 \times 10^6$ single-precision elements, $D = 12N = 24$ MB), the analytical model estimates a CPU-only latency of 0.9016 ms and a direct-NMC latency of 0.7742 ms. Substituting these values into (5) gives a speedup of $0.9016/0.7742 \approx 1.16\times$, which corresponds to a latency reduction of approximately 14% relative to the host baseline. The comparison assumes that both times represent the same workload and include comparable execution overheads.

Fig. 7 compares the end-to-end latency of the host-CPU path with that of the direct-NMC offload for the benchmark loop. **Inference:** for this memory-bound kernel, relocating computation close to memory removes enough off-chip transfer time to give a net improvement of about 0.127 ms even after launch, transfer and synchronization overheads are included. The benefit is modest at this size because the host-to-device transfer remains on the critical path (Fig. 9), and it grows with problem size (Fig. 8) and with link bandwidth (Fig. 12).

The absolute figures behind Fig. 7 follow directly from the cost model. The host path completes in 0.9016 ms, of which 0.8571 ms (95.1%) is off-chip transfer, whereas the direct-NMC path completes in 0.7742 ms, a reduction of 0.1274 ms or about 14.1%, which is the speedup of 1.16× reported in Table 3. The gain is limited because the near-memory path introduces costs that the host path does not pay, namely host-to-device transfer, layout conversion and synchronization (Fig. 9), so only the part of the transfer time that is replaced by cheaper internal bank access is recovered. The speedup is therefore a property of the operating point and not of the kernel alone: it moves with input size (Fig. 8), link bandwidth (Fig. 12) and data layout (Fig. 13). For this reason every speedup in this paper is stated together with its configuration, and, because the cost model is analytical, the values should be read as estimates of relative behavior.

2) Input-Size Scaling and Crossover Behavior

The input-size experiment examines how the relative costs of host and near-memory execution change with workload size, from 70K to 5.6M elements. NMC-OPT identifies an approximate crossover at 700K elements. Below this point, fixed launch, setup and synchronization overheads favor CPU execution; above it, the near-memory path benefits from the higher internal bandwidth.

Fig. 8 traces execution cost against input size for both paths, and Fig. 5 shows the same data as the signed difference $T_{\text{CPU}} - T_{\text{NMC}}$. **Inference:** the preferred strategy is size-dependent. At 70K elements the near-memory path is about three times slower (≈0.10 ms versus ≈0.03 ms) because a size-independent overhead dominates a very small amount of work, whereas the CPU cost grows linearly with N. The two curves meet near 700K elements (≈0.32 ms), and the gap then widens steadily: at 2.0M elements the estimates are 0.90 ms (CPU) and 0.77 ms (NMC), and at 5.6M elements they are about 2.52 ms and 2.04 ms, a speedup of about 1.24×. The near-memory advantage therefore scales with workload size, but it saturates at roughly 1.28× for the direct offload because both paths grow linearly with N once the fixed overhead is amortized. A compiler that always offloads would lose on small inputs, which is the case the cost-based decision of Algorithm 1 is

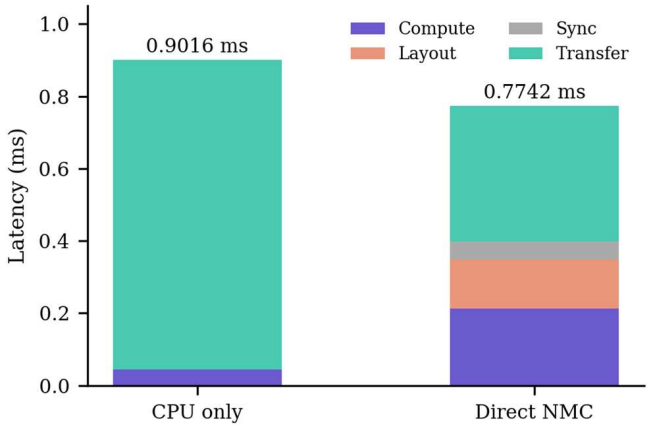


FIGURE 9. Breakdown of execution latency by component.

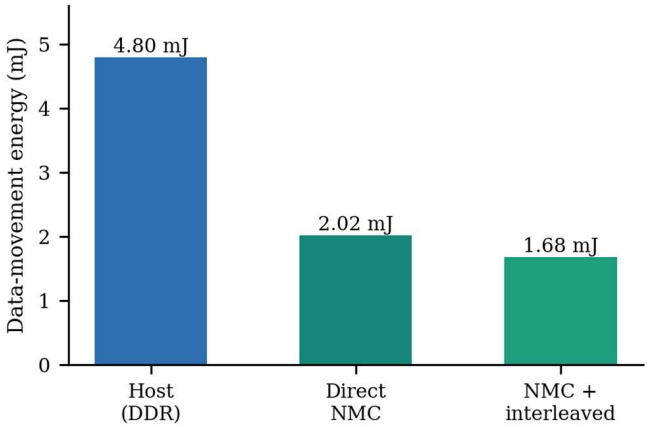


FIGURE 10. Data-movement energy across execution strategies.

designed to avoid.

The crossover has a direct consequence for how the compiler is used. When the input size is known at compile time, NMC-OPT can evaluate the cost model once and emit either the host or the near-memory version. When the size is known only at run time, the compiler can emit both versions together with a size guard set at the predicted crossover of about 700K elements, so that small inputs remain on the host and large inputs are offloaded. The crossover is not a constant: it moves to larger sizes when the fixed launch and transfer overhead grows (for example, at a lower link bandwidth) and to smaller sizes when that overhead shrinks, which is why the threshold is derived from the platform parameters rather than hard-coded. Above the crossover the absolute saving continues to grow, from about 0.13 ms at 2.0M elements to about 0.48 ms at 5.6M elements.

3) Latency Component Breakdown

The latency-breakdown experiment separates execution time into computation, data layout, synchronization, and data transfer. The breakdown compares the host-CPU and near-memory paths to show which components dominate total execution time.

Fig. 9 decomposes each path at 2.0M elements. The host path consists of 0.0444 ms of computation (4.9%) and 0.8571 ms of off-chip memory transfer (95.1%), giving 0.9016 ms. The near-memory path consists of 0.2125 ms of computation (27.4%), 0.3750 ms of host-to-device transfer (48.4%), 0.1367 ms of data-layout conversion (17.7%) and 0.0500 ms of synchronization (6.5%); these components sum to the 0.7742 ms reported in Fig. 7. **Inference:** the CPU total is almost entirely a data-movement cost, which is the opportunity that near-memory execution exploits: the transfer term falls from 0.857 ms to 0.375 ms (a 56% reduction). The near-memory path pays back part of this saving through layout conversion and synchronization (0.187 ms together) that the host does not incur, and its computation term is larger because it includes internal bank access. Interconnect transfer nevertheless remains the single largest term of the near-memory path, which identifies the host-to-device link, rather than computation, as the primary residual bottleneck and the main target for further optimization.

4) Data-Movement Energy Consumption

As shown in Fig. 10, data-movement energy is estimated as the transferred volume multiplied by a per-byte energy: about 200 pJ/byte (25 pJ/bit) for off-chip DDR transfers, about 84 pJ/byte for near-memory execution (short internal 3D-TSV links at about 28 pJ/byte plus the host link at about 56 pJ/byte), and about 70 pJ/byte with bank-interleaved placement. For $D = 24$ MB these give 4.80 mJ, 2.02 mJ and 1.68 mJ respectively, i.e., an average data-movement power of about 5.3 W for the host (4.80 mJ over 0.9016 ms) and about 2.6 W for direct NMC (2.02 mJ over 0.7742 ms). Applying (8) gives energy reductions of 58% for direct NMC and 65% for bank-interleaved NMC relative to the host. **Inference:** because near-memory execution shortens the physical distance data must travel, it saves energy as well as time, and the saving depends on the bytes moved and the per-byte transfer energy rather than on arithmetic, so it is approximately constant across the benchmark suite. The interleaved layout contributes an additional 0.34 mJ saving over direct offload, showing that data placement affects energy as well as latency.

5) Arithmetic Intensity and Roofline Analysis

Fig. 11 places the benchmark kernel on a classical roofline. The host ceiling is $\min(45\ \text{GFLOP/s}, 28\ \text{GB/s} \times I)$ and the near-memory ceiling is $\min(32\ \text{GFLOP/s}, 160\ \text{GB/s} \times I)$, with ridge points at about 1.6 and 0.2 FLOP/byte, respectively. The kernel's arithmetic intensity, from (6), is $I = 1/12 \approx 0.083$ FLOP/byte, where the attainable throughput is 2.32 GFLOP/s on the host and 13.28 GFLOP/s near memory. **Inference:** because 0.083 FLOP/byte lies far to the left of both ridge points, the kernel is strongly memory-bandwidth bound and its throughput is set by data supply rather than by compute capability. This is the regime in which higher internal memory bandwidth helps most: the roofline ceiling is about $5.7\times$ higher near memory. The end-to-end speedup is only $1.16\times$ because the roofline ignores launch, layout and host-to-device transfer, which the cost model of Section III-F adds back; the roofline therefore bounds the potential benefit rather than predicting it.

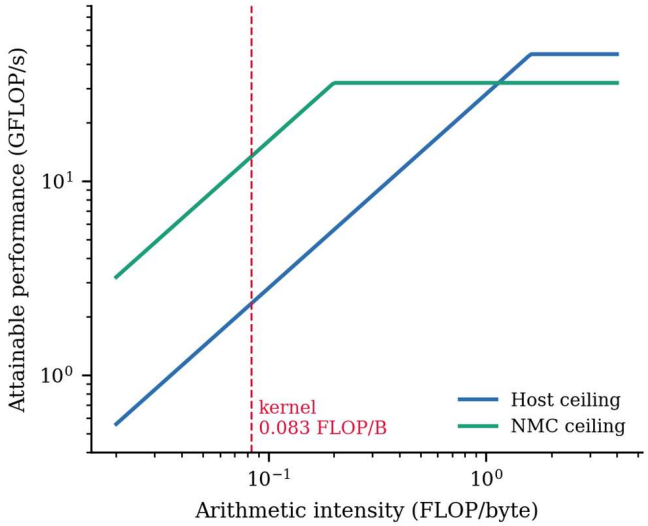


FIGURE 11. Roofline model and kernel operating point.

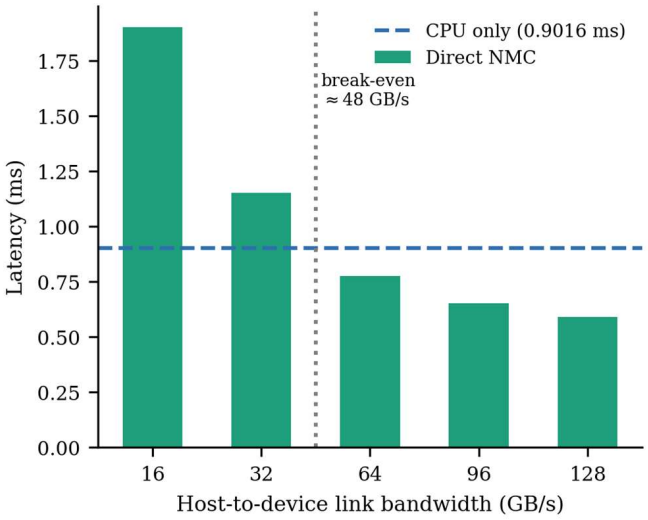


FIGURE 12. Interconnect bandwidth sensitivity.

6) Interconnect Bandwidth Sensitivity

Fig. 12 sweeps the host-to-device interconnect bandwidth from 16 to 128 GB/s while the host baseline remains at 0.9016 ms. The near-memory latency falls from about 1.90 ms at 16 GB/s to 1.15, 0.77, 0.65 and 0.59 ms at 32, 64, 96 and 128 GB/s, so the speedup rises from 0.47× to 0.78×, 1.16×, 1.39× and 1.54×. **Inference:** near-memory latency falls steadily as link bandwidth rises. Below a break-even bandwidth of about 48 GB/s the offload is slower than the host, and above it the offload wins, so the attainable speedup is highly sensitive to interconnect provisioning. The near-memory benefit therefore depends on the assumed link bandwidth; the 1.16× of Fig. 7 corresponds to the 64 GB/s default link.

7) Bank-Conflict and Layout Analysis

Fig. 13 compares execution latency across four memory layouts: an unaligned naive-DRAM layout (1.045 ms), a block-padded layout (0.890 ms), a bank-interleaved layout (0.774 ms), and a PIM-native SIMD-packed layout (0.712 ms). Relative to the bank-interleaved layout, these correspond to channel-collision stall factors of about 1.35, 1.15, 1.00 and 0.92 in the cost model. **Inference:** the unaligned layout is about 35% slower than bank-interleaved striping (equivalently, striping removes about 26% of the unaligned latency) and the block-padded layout is about 15% slower, because collisions in the memory channel stall concurrent requests. The SIMD-packed layout is about 8% faster than interleaving, which indicates the further headroom available from device-specific packing. Data placement therefore has a material effect on near-memory performance in addition to the decision to offload, and the 35% figure is a modelled stall penalty rather than a measured value.

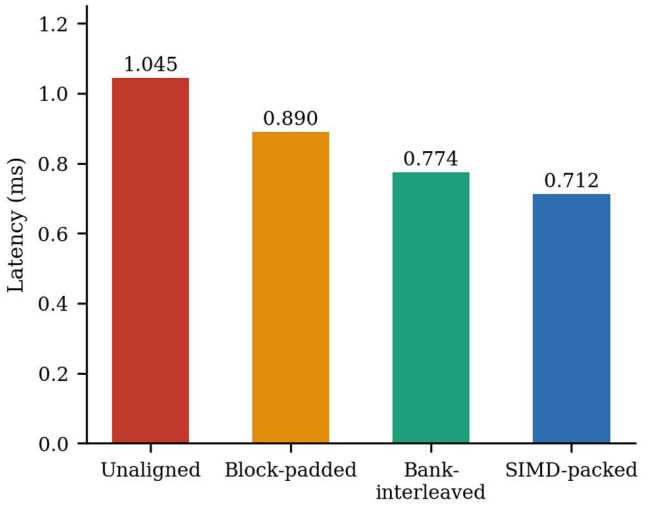


FIGURE 13. Memory-layout latency comparison.

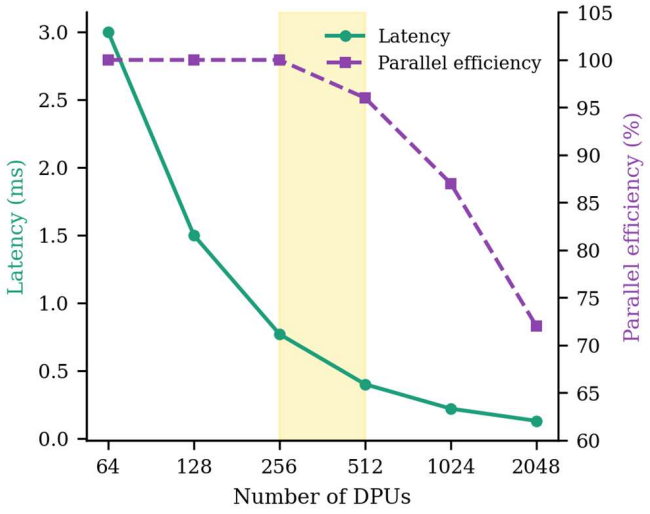


FIGURE 14. DPU scaling and parallel efficiency.

8) DPU Scaling and Parallel Efficiency

$$\eta_p = \frac{T_1}{p\,T_p} \tag{9}$$

Here, η_p denotes the parallel efficiency on p processing units, T_1 is the execution time of the reference configuration on a

single processing unit, and T_p is the execution time on p processing units. Because the sweep starts at 64 DPUs, T_1 is taken as $64 \times T_{64}$ ($\approx$192 ms), which assumes ideal scaling up to the first measured point.

Parallel efficiency decreases as the unit count grows because serial reduction and transfer overheads become relatively more significant. The sweet spot of 256–512 DPUs corresponds to the region where these overheads begin to outweigh the gains from additional parallelism.

Fig. 14 plots execution latency and parallel efficiency as the number of processing units scales from 64 to 2048. Latency falls from about 3.0 ms at 64 DPUs to 1.5, 0.77, 0.40, 0.22 and 0.13 ms at 128, 256, 512, 1024 and 2048 DPUs, while efficiency stays at 100% up to 256 DPUs and then declines to about 96%, 87% and 72%. **Inference:** latency drops sharply through the mid-range while efficiency remains near-ideal; doubling from 256 to 512 DPUs still gives a 1.9× reduction, but quadrupling from 512 to 2048 DPUs gives only about 3.0×, so the last doubling yields diminishing returns. The curves therefore identify a practical sweet spot at 256–512 DPUs, consistent with an Amdahl-style limit. These curves are produced by the same analytical model; no UPMEM hardware or simulator was executed, so they should be read as scaling trends of the model rather than as measurements.

E. COMPARATIVE ANALYSIS WITH RELATED WORK

Table 4 compares the scope, optimization approaches, platforms, and reported results of CAIRO, IMTP, and NMC-OPT.

As Table 4 shows, CAIRO and IMTP respectively target instruction-level offloading and search-based code generation on specific platforms, whereas NMC-OPT occupies a complementary position: static source-level analysis with an explicit analytical CPU-versus-NMC cost model that estimates crossover thresholds before execution. The comparison also makes clear that the NMC-OPT figures reported here are analytical or prototype estimates rather than measured hardware results.

F. DISCUSSION AND LIMITATIONS

The reported results suggest that NMC-OPT may reduce execution cost for selected memory-intensive workloads when the transfer and launch overheads are outweighed by the benefits of near-memory processing. The input-size and interconnect sensitivity experiments are particularly relevant because they examine when the preferred execution strategy changes.

The strength of these conclusions is limited by the current prototype. All reported figures are analytical estimates produced by the cost model rather than hardware or cycle-accurate-simulation measurements, so their absolute magnitudes should be read as model outputs rather than observed values. The energy figures in particular are analytical estimates whose absolute calibration against a specific device has not yet been established, and the cost-model parameters have not been tuned to a physical target. The configuration files, workload definitions, and calculation procedures used to produce the estimates are maintained with the prototype and are available on request.

G. BENCHMARK KERNEL CHARACTERISTICS

The evaluation uses a small suite of representative data-parallel kernels, each chosen to exercise a different memory-access and computation pattern. Vector Add and Fused AXPY are streaming element-wise kernels that read one or more input arrays and write a single output array, exhibiting low arithmetic intensity and highly regular access. Element-wise Vector Multiply follows the same streaming pattern with a different arithmetic operation. Reduction reads a large input and collapses it to a scalar or small output, combining streaming access with a dependency across iterations. ReLU is a unary activation kernel that applies a simple threshold element-wise, and GEMV, the general matrix-vector multiplication, introduces a dense linear-algebra pattern in which each output element depends on a row of the matrix and the full input vector.

These kernels were selected because their performance is governed primarily by data movement rather than by arithmetic throughput, which is precisely the regime in which near-memory execution is expected to help. Their regular and well-understood access patterns make the analytical cost estimates easier to interpret and the experiments easier to reproduce. Collectively they span the element-wise, reduction, activation, and matrix-vector families, allowing the execution-placement decisions of the framework to be examined across distinct memory-behaviour classes rather than on a single workload.

H. ILLUSTRATIVE COST-MODEL CALCULATION

To illustrate how the cost model and metrics are applied, consider the example values for the benchmark loop. Using the speedup definition in (5) with a host latency of approximately 0.90 ms and a near-memory latency of approximately 0.77 ms gives a speedup of about 1.16 times, corresponding to a latency reduction of roughly 14% relative to the host baseline. The arithmetic intensity in (6) for this kernel is approximately 0.083 FLOP per byte, which places it firmly in the memory-bound region of the roofline and explains why additional internal bandwidth, rather than additional compute capability, is the dominant lever on performance.

Applying the energy-reduction expression in (8) to the data-movement energy figures of about 4.80 mJ for the host baseline and about 2.02 mJ for near-memory execution yields a reduction of approximately 58% (65% for the interleaved layout at 1.68 mJ). This magnitude is consistent with the much shorter physical distance that data travels inside the memory device compared with transfers across the off-chip interconnect. These calculations are illustrative and inherit the assumptions of the underlying cost model; they are intended to show how the framework converts estimated quantities into comparable metrics rather than to serve as validated hardware measurements.

I. THREATS TO VALIDITY AND REPRODUCIBILITY

Several factors should be considered when interpreting the results. The principal threat to construct validity is the dependence of all reported figures on the analytical cost model; if its parameters are not calibrated to a specific target, the absolute latencies and energies should be read as estimates rather than

TABLE 4. Comparison with CAIRO and IMTP/ATiM

Dimension	CAIRO [8]	IMTP/ATiM [13]	NMC-OPT (proposed work)
Main focus	Compiler-assisted, instruction-level PIM offloading using HMC-atomic instructions, evaluated on graph workloads	Search-based code generation for in-memory tensor programs on UPMEM	Analytical cost modelling, IR lowering, and bank-level data placement for CPU-vs-NMC decisions
Optimization approach	Profiling-based compile-time decision model: eligibility tests, LLC miss-ratio analysis, and bandwidth-savings analysis. Machine constants come from micro-benchmarks and linear regression.	Autotuning in TVM: evolutionary search with a learned cost model over a joint host and kernel schedule space. Adds PIM-aware passes (DMA-aware boundary-check elimination, loop-bound tightening, invariant branch hoisting).	Cost-based strategy selection (CPU vs NMC) with loop-transformation candidates and 16-bank striping
Execution platform	Simulated CPU+HMC 2.0 (16 out-of-order cores, 8 GB HMC with 32 vaults) and GPU+HMC (15 SMs). Uses MacSim, VaultSim, and SST.	Real UPMEM server: dual-socket Xeon Gold 5220R, 2048 DPUs (32 ranks), TVM 0.13.0, UPMEM SDK 2021.3.0	Analytical cost model in a software prototype (Python 3 / FastAPI backend with pytest unit tests; React 19 / Vite 8 frontend); no physical or cycle-accurate NMC target is used.
Latency	Up to 2× speedup on PIM-beneficial workloads and no slowdown on the others. 63 workloads (27 CPU, 36 GPU).	Average 3.05× over PrIM, 1.48× over PrIM(E), 1.67× over PrIM+search, and 3.3× over SimplePIM. Up to 8.21× on GPT-J layers.	0.90 ms (CPU-only) vs 0.77 ms (direct NMC), a derived speedup of approximately 1.16×
Energy	Not reported quantitatively	Not reported	4.80 mJ (host DDR), 2.02 mJ (direct NMC), 1.68 mJ (interleaved NMC); 58% / 65% lower
Research gap addressed	Instruction-level and fixed-function scope, profiling dependence, and simulated hardware. Does not make whole-kernel offloading decisions.	UPMEM-specific, needs hardware measurements for the search, and provides no compact analytical CPU-vs-NMC explanation	NMC-Opt Studio uses static source-level analysis and analytical cost modeling to estimate workload crossover thresholds and guide efficient NMC offloading strategy decisions before execution.

measurements. The main threat to internal validity is the distinction between simulated, analytically derived, and measured values, which is stated explicitly here: every value reported in this paper is analytical. The same data-movement boundary is used for the host and near-memory paths in all comparisons.

The main threat to external validity is generalization beyond the kernels and configurations studied here. The benchmark suite is deliberately small and regular, so the crossover thresholds, bandwidth-sensitivity trends, and layout benefits reported for these workloads may differ for irregular or control-heavy kernels, for other bank configurations, or for alternative interconnect technologies. The framework is intended to expose its assumptions so that such differences can be re-evaluated rather than assumed away.

To support reproducibility, each experiment records the kernel source, input size, selected execution strategy, transformation and layout configuration, cost-model parameters, and the environment in which the estimates were produced. Reporting these details alongside the raw result files allows the analytical predictions to be checked against independent measurements and allows other researchers to reproduce the reported crossover points and sensitivity trends under comparable assumptions.

J. DEPLOYMENT AND INTEGRATION CONSIDERATIONS

Beyond its analytical role, NMC-OPT is designed to be usable within a practical development workflow. The framework exposes two complementary interfaces described earlier: an interactive web dashboard for exploring kernels, inspecting the intermediate representation, and viewing optimization verdicts, and a command-line interface suitable for automation and scripting. This separation allows the same analysis engine to be driven interactively during exploration and non-interactively as part of a larger build or evaluation process.

Because the decision procedure is static and runs ahead of execution, it can in principle be invoked as an additional analysis pass within an existing compiler toolchain or continuous-integration pipeline, flagging kernels whose predicted crossover behaviour favours near-memory execution. The modular backend keeps source validation, workload analysis, cost modelling, candidate evaluation, and reporting as separate stages, so an integrator can reuse the cost model and strategy-selection logic independently of the user interface.

Two practical considerations follow from this design. First, the quality of any integration depends on supplying cost-model parameters that reflect the target platform, since the analysis is only as accurate as its calibration. Second, the reporting stage preserves the full decision record, including the inputs, estimates, selected strategy, and configuration, so that automated decisions remain auditable and can be revisited when hardware or workload assumptions change. These considerations keep the framework useful as a decision-support tool while making clear that its recommendations complement, rather than replace, measurement on the target system.

V. CONCLUSION AND SCOPE FOR FUTURE WORK

This paper presents NMC-OPT, a compiler-assisted framework that automates execution-cost estimation, optimization, and offloading decisions for near-memory computing workloads. The framework combines analytical cost modelling, intermediate representation analysis, loop fusion and tiling transformations, and bank-level data placement.

Existing PIM frameworks and compiler optimization systems address important aspects of near-memory execution,

including instruction-level offloading, tensor scheduling, and workload-specific acceleration [8], [13], [14], [22]. However, execution overheads, memory layout, and workload characteristics continue to influence whether near-memory execution is beneficial. The proposed approach integrates these concerns into a unified optimization workflow. Its principal advantage is the ability to compare predicted execution costs before selecting a host or near-memory execution strategy. The framework also provides a basis for evaluating loop transformations and data-placement choices.

NMC-OPT draws on MacSim, DRAMSim2, SST, and Apache TVM as design references and is implemented as a FastAPI/Vite application [15], [27], [28], [29]. The quantitative results reported here are produced by the framework's analytical cost model, and integrating calibrated simulator or hardware measurements is left to future work.

For the example kernel, the analytical model estimates a 1.16× end-to-end speedup for Vector Add at two million elements, and between 1.23× and 1.50× for the other kernels of the suite (1.50× for ReLU with loop fusion and for GEMV with bank tiling). Relative to an unaligned always-near-memory placement, bank-interleaved placement is about 1.35× faster. The model estimates a crossover near 700K elements, a link-bandwidth break-even near 48 GB/s, about 58% lower data-movement energy (65% with interleaving), and a parallel-efficiency sweet spot of 256–512 DPUs. These are analytical estimates from the prototype and remain to be validated against hardware or cycle-accurate simulation, including a clear separation of measured, simulated, and analytical values.

Future work will investigate the integration of graph-level intermediate representations, including TVM Relax, to support more complex computational graphs and automated workload partitioning. Additional research may examine dynamic workload sizes, more general memory-access patterns, and cost-model calibration across heterogeneous platforms. Extending the framework to emerging CXL-based memory systems and other near-memory technologies is another possible direction. Finally, systematic validation against existing PIM frameworks and reproducible benchmarks will be essential for establishing the generality and effectiveness of NMC-OPT.

AUTHOR CONTRIBUTIONS

Malisetty Sai Nikhit: Software (Full-Stack System Architecture, Frontend and Backend Development), Data Curation (Visual Analytics and Interactive Data Presentation), System Optimization, and Writing (Manuscript Review and Editing).

Dhayapule Kartikesh Rao: Software (UI/UX Design and Implementation, Frontend Workbench Development), Validation (Software Testing and Verification), Analytics and Graph Implementation, and Writing (Manuscript Drafting and Preparation).

Metta Venkata Surya Sai Srikar: Conceptualization and Methodology (System Architecture, Cost Modelling, Memory Layout), Software (Website Implementation and Backend API Integration), Validation (Software Testing and Verification), and Formal Analysis (Result and Parameter Analysis).

Aahaan Kothari: Mathematical Formulation, Cost Modelling, Memory-Traffic Analysis, Optimization Evaluation, Data-Movement Cost Analysis, Interconnect and Energy-Dissipation Validation, Research Paper Analysis.

Govind Bhatia: Formal Analysis, Software (Intermediate Representation, Loop Fusion, Tiling), Data-Dependency Analysis, Memory-Access Analysis, Cost-Model Evaluation, Investigation, Strategy Evaluation, Research Paper Analysis.

All authors reviewed, revised, and approved the final manuscript.

ACKNOWLEDGEMENTS

The authors sincerely thank Prof. Bhuvaneswari M, of the Department of Computer Science and Engineering, Vellore Institute of Technology, Vellore, for her valuable guidance, constructive feedback, and encouragement throughout the development of this work. Her suggestions and support contributed to improving the design, implementation, and presentation of the proposed framework.

We gratefully acknowledge Vellore Institute of Technology, Vellore, and in particular its Department of Computer Science and Engineering, for providing the academic environment, computing facilities, and institutional support that made this work possible.

We also acknowledge the open-source software ecosystem on which the NMC-OPT prototype is built. The system relies on Python for the analytical cost engine and backend, TypeScript and JavaScript for the application logic, and HTML and CSS for the web dashboard, together with the associated open-source frameworks and libraries. The availability of these freely accessible tools was essential to implementing, testing, and presenting the proposed framework.

Finally, we acknowledge the benchmark kernels and datasets used in this study—including the Vector Add, Reduction, ReLU, GEMV, Fused AXPY, and element-wise multiply workloads—which were central to characterizing memory-bound behaviour and evaluating the CPU-versus-NMC execution decisions reported here. Representative, well-understood workloads of this kind are important because they make the analysis reproducible and allow the framework's recommendations to be assessed against established computational patterns.

DECLARATION OF GENERATIVE AI USE

During the preparation of this manuscript, the authors used Claude (Anthropic) to assist with revising and editing the text, redrawing figures, and checking the consistency of reported values. The authors reviewed and edited all AI-assisted material and take full responsibility for the content of the published article.

FUNDING

This research received no external funding from any public, commercial, or non-profit funding agency.

COMPETING INTERESTS

The authors declare that they have no competing financial or personal conflicts of interest.

DATA AND CODE AVAILABILITY

The source code, compiler passes, analytical cost engines, benchmark kernels, and reproduction scripts that support the findings of this study are maintained in a private repository (https://github.com/MSN-1307/NMC-OPT-CD) and are not publicly accessible.

Access will be granted by the corresponding author to interested researchers upon reasonable request, subject to a suitable usage agreement. The same applies to the full datasets, generated result files, configuration files, and any supplementary material associated with the study, which will likewise be shared on request to support verification and reproduction of the reported results.

REFERENCES

[1] O. Mutlu, S. Ghose, J. Gómez-Luna, and R. Ausavarungnirun, "A modern primer on processing in memory," in *Emerging Computing: Looking Beyond Moore and Von Neumann*, Berlin, Germany: Springer, 2022, pp. 171–243.

[2] B. Y. Cho, J. Jung, and M. Erez, "Accelerating bandwidth-bound deep learning inference with main-memory accelerators," in *Proc. Int. Conf. High Perform. Comput., Netw., Storage Anal. (SC)*, Nov. 2021, Art. no. 44.

[3] G. F. Oliveira et al., "MIMDram: An end-to-end processing-using-DRAM system for high-throughput MIMD computing," in *Proc. IEEE Int. Symp. High-Perform. Comput. Architecture (HPCA)*, Mar. 2024, pp. 186–203.

[4] F. Devaux, "The true processing in memory accelerator," in *IEEE Hot Chips 31 Symp. (HCS)*, Aug. 2019, pp. 1–24.

[5] J. H. Kim et al., "Aquabolt-XL: Samsung HBM2-PIM with in-memory processing for ML accelerators and beyond," in *IEEE Hot Chips 33 Symp. (HCS)*, Aug. 2021, pp. 1–26.

[6] B. Hyun, T. Kim, D. Lee, and M. Rhu, "Pathfinding future PIM architectures by demystifying a commercial PIM technology," in *Proc. IEEE Int. Symp. High-Perform. Comput. Architecture (HPCA)*, Mar. 2024, pp. 263–279.

[7] J. Gómez-Luna et al., "Evaluating machine learning workloads on memory-centric computing systems," in *Proc. IEEE Int. Symp. Perform. Anal. Syst. Softw. (ISPASS)*, Apr. 2023, pp. 35–49.

[8] R. Hadidi, L. Nai, H. Kim, and H. Kim, "CAIRO: A compiler-assisted technique for enabling instruction-level offloading of processing-in-memory," *ACM Trans. Architecture Code Optim.*, vol. 14, no. 4, Art. no. 48, pp. 48:1–48:25, Dec. 2017.

[9] M. A. Ibrahim, M. Islam, and S. Aga, "Balanced data placement for GEMV acceleration with processing-in-memory," in *Proc. Int. Symp. Memory Syst. (MEMSYS)*, Oct. 2024, pp. 112–125.

[10] K. Gogineni et al., "SwiftRL: Towards efficient reinforcement learning on real processing-in-memory systems," in *Proc. IEEE Int. Symp. Perform. Anal. Syst. Softw. (ISPASS)*, May 2024, pp. 217–229.

[11] J. Ahn, S. Yoo, O. Mutlu, and K. Choi, "PIM-enabled instructions: A low-overhead, locality-aware PIM architecture," in *Proc. ACM/IEEE 42nd Annu. Int. Symp. Comput. Architecture (ISCA)*, Jun. 2015, pp. 336–348.

[12] L. Nai, R. Hadidi, J. Sim, H. Kim, P. Kumar, and H. Kim, "GraphPIM: Enabling instruction-level PIM offloading in graph computing frameworks," in *Proc. IEEE Int. Symp. High-Perform. Comput. Architecture (HPCA)*, Feb. 2017, pp. 457–468.

[13] Y. Shin, D. Kang, and H. Sung, "ATiM: Autotuning tensor programs for processing-in-DRAM," in *Proc. 52nd Annu. Int. Symp. Comput. Architecture (ISCA)*, Jun. 2025, doi: 10.1145/3695053.3731096. (arXiv:2412.19630, first released as "IMTP: Search-based code generation for in-memory tensor programs.")

[14] J. Chen, J. Gómez-Luna, I. El Hajj, Y. Guo, and O. Mutlu, "SimplePIM: A software framework for productive and efficient processing-in-memory," in *Proc. Int. Conf. Parallel Architectures Compiler Tech. (PACT)*, Oct. 2023, pp. 99–111.

[15] T. Chen et al., "TVM: An automated end-to-end optimizing compiler for deep learning," in *Proc. 13th USENIX Symp. Oper. Syst. Design Implement. (OSDI)*, Oct. 2018, pp. 578–594.

[16] J. Gómez-Luna et al., "Benchmarking a new paradigm: Experimental analysis and characterization of a real processing-in-memory system," *IEEE Access*, vol. 10, pp. 52565–52608, May 2022.

[17] A. A. Khan, H. Farzaneh, K. F. A. Friebel, C. Fournier, L. Chelini, and J. Castrillon, "CINM (Cinnamon): A compilation infrastructure for heterogeneous compute in-memory and compute near-memory paradigms," in *Proc. ACM Int. Conf. Architectural Support Program. Lang. Oper. Syst. (ASPLOS)*, Mar. 2025, pp. 31–46.

[18] C. Giannoula et al., "SparseP: Efficient sparse matrix vector multiplication on real processing-in-memory architectures," in *Proc. IEEE Comput. Soc. Annu. Symp. VLSI (ISVLSI)*, Jul. 2022, pp. 288–291.

[19] S. Lee et al., "A 1ynm 1.25V 8Gb 16Gb/s/pin GDDR6-based accelerator-in-memory supporting 1TFLOPS MAC operation," in *Proc. IEEE Int. Solid-State Circuits Conf. (ISSCC)*, vol. 65, Feb. 2022, pp. 1–3.

[20] C. Lim et al., "Design and analysis of a processing-in-DIMM join algorithm: A case study with UPMEM DIMMs," *Proc. ACM Manag. Data*, vol. 1, no. 2, Art. no. 147, pp. 1–27, Jun. 2023.

[21] B. Zheng et al., "DietCode: Automatic optimization for dynamic tensor programs," in *Proc. Conf. Mach. Learn. Syst. (MLSys)*, vol. 4, Aug. 2022, pp. 848–863.

[22] L. Zheng et al., "Ansor: Generating high-performance tensor programs for deep learning," in *Proc. 14th USENIX Symp. Oper. Syst. Design Implement. (OSDI)*, Oct. 2020, pp. 863–879.

[23] S. Feng et al., "TensorIR: An abstraction for automatic tensorized program optimization," in *Proc. 28th ACM Int. Conf. Architectural Support Program. Lang. Oper. Syst. (ASPLOS)*, Mar. 2023, pp. 804–817.

[24] A. Boroumand et al., "LazyPIM: An efficient cache coherence mechanism for processing-in-memory," *IEEE Comput. Architecture Lett.*, vol. 16, no. 1, pp. 46–50, Jan.–Jun. 2017.

[25] C. Lattner et al., "MLIR: Scaling compiler infrastructure for domain specific computation," in *Proc. IEEE/ACM Int. Symp. Code Gener. Optim. (CGO)*, Feb. 2021, pp. 2–14.

[26] J. Lin et al., "SongC: A compiler for hybrid near-memory and in-memory many-core architecture," *IEEE Trans. Comput.*, vol. 73, no. 10, pp. 2420–2433, Oct. 2024.

[27] H. Kim, R. Hadidi, and L. Nai, "MacSim: A cycle-level microarchitecture simulator for heterogeneous systems," Georgia Institute of Technology, Atlanta, GA, USA, Tech. Rep., 2018.

[28] P. Rosenfeld, E. Cooper-Balis, and B. Jacob, "DRAMSim2: A cycle accurate memory system simulator," *IEEE Comput. Architecture Lett.*, vol. 10, no. 1, pp. 16–19, Jan.–Jun. 2011.

[29] A. Rodrigues et al., "The Structural Simulation Toolkit," *ACM SIGMETRICS Perform. Eval. Rev.*, vol. 38, no. 4, pp. 37–42, Mar. 2011.

[30] Y. Shin, J. Park, S. Cho, and H. Sung, "PIMFlow: Compiler and runtime support for CNN models on processing-in-memory DRAM," in *Proc. ACM/IEEE Int. Symp. Code Gener. Optim. (CGO)*, Feb. 2023, pp. 249–262.



MALISETTY SAI NIKHIT is currently pursuing the B.Tech. degree in Computer Science and Engineering at Vellore Institute of Technology, Vellore, India. His academic interests include full-stack system design, visual analytics, and compiler-assisted optimization. He is interested in developing software systems that combine effective architectural design with intuitive visual representations. His broader interests include interactive system modelling, software architecture visualization, and tools that help developers understand complex program structures. He is particularly interested in exploring how visual analytics and compiler techniques can support the analysis and optimization of modern computing systems.



METTA VENKATA SURYA SAI SRIKAR is currently pursuing the B.Tech. degree in Computer Science and Engineering at Vellore Institute of Technology, Vellore, India. His research interests include analytical cost modelling, memory-system design, and backend systems. He is interested in studying the relationship between computation, data movement, and hardware resource utilization. His broader interests include developing analytical models for estimating execution costs, understanding memory-related performance constraints, and evaluating system-level trade-offs. He is particularly interested in how memory organization and data-access patterns influence the efficiency of software and computer architectures.



GOVIND BHATIA is currently pursuing the B.Tech. degree in Computer Science and Engineering at Vellore Institute of Technology, Vellore, India. His research interests include intermediate representations, loop transformations, and data-dependence analysis. He is interested in compiler techniques that analyse program structure and determine the conditions under which code transformations can be applied safely and efficiently. His areas of interest include static program analysis, dependency-aware optimization, and memory-access analysis. He is particularly interested in loop transformations such as fusion and tiling, and in their potential to improve data locality and execution efficiency in near-memory computing architectures.



DHAYAPULE KARTIKESH RAO is currently pursuing the B.Tech. degree in Computer Science and Engineering at Vellore Institute of Technology, Vellore, India. His research interests include software engineering, compiler design and optimization, software testing, and performance analysis. He is interested in software architecture, code optimization, and developing reliable, efficient, and maintainable applications. His broader interests include debugging, software quality assurance, and compiler construction. He values analytical thinking, problem-solving, attention to detail, and collaborative development to build high-quality software systems while continuously exploring effective engineering practices.



AAHAAN KOTHARI is currently pursuing the B.Tech. degree in Computer Science and Engineering at Vellore Institute of Technology, Vellore, India. His research interests include mathematical modelling of memory traffic and energy, and performance analysis. He is interested in using quantitative methods to study the effects of data movement and memory accesses on execution time and energy consumption. His broader interests include performance characterization, analytical evaluation of computing systems, and the identification of trade-offs between computational efficiency and resource utilization. He aims to develop a deeper understanding of how mathematical models can support the evaluation and comparison of different system designs.